Dade Kutzera
CSCI 430/530
Sarnath Ramnath
10.07.2026

Receive Shipment

Business Process:

Clerk (trusted actor) makes a request to receive a shipment and provides the product ID and the quantity received. If the product is found, the system goes through the waitlist and, for each client, completes an order for that product with the specified quantity (unless we run out, in which case unfilled qty remains on the waitlist), and debits the cost to the client's account. When the waitlist is processed, the remaining amount is added to the product's stock.

Classes Used by This Feature

This feature uses the Stage 1 classes Warehouse, ClientList, ProdList, Client, Product, and WaitlistItem, plus the Order class introduced in Process Client's Order (7 classes). WishlistItem is not used by this feature. The user interface talks only to Warehouse.

Class	Responsibilities added by this feature
Warehouse (facade)	Coordinates receiveShipment(productId, receivedQty). Finds the product through ProdList, walks the product's waitlist, creates a new Order for each client whose request is filled, debits that client, updates or removes the WaitlistItem, adds any remaining quantity to stock, and returns a shipment summary to the user interface. Also provides getProduct(productId), getWaitlist(productId), getClient(clientId), and getOrders(clientId) for the verification queries.
ClientList	search(clientId) returns the Client or null. Used by Query Client Details and Query Client Orders. receiveShipment does not need it, because each WaitlistItem already holds its Client reference.
ProdList	search(productId) returns the Product receiving the shipment, or null.
Client	Holds account balance and orders. Uses debit(amount), addOrder(order), getBalance(), and getOrders(). Client is charged when waitlisted items are filled.
Product	Holds stock quantity, price, and waitlist. Uses getStock(), getPrice(), getWaitlist(), addStock(qty), and removeFromWaitlist(waitlistItem).
WaitlistItem	Holds a client reference and the quantity still requested. Uses getClient(), getQuantity(), and setQuantity(qty) when a request is only partly filled.

Class	Responsibilities added by this feature
Order	Records one filled waitlist request. A new Order(client, date) is created for each client served. Uses addItem(product, qty, price), getItems(), and getTotal(). The unit price is copied into the order line so later price changes do not alter past orders.

Question 1: Query Implementation:

Receiving a shipment can change four kinds of stored data: the product's stock, the product's waitlist, the client's balance, and the client's orders. The queries below are run before and after processing the shipment so the results can be compared.

Data Stored In The Warehouse System That May Be Modified	Query Before Testing The Operation	Query After Testing The Operation	How The Before And After Queries Verify The Operation Was Done Correctly
Product stock	Run Query Product Details and note the current stock quantity.	Run Query Product Details again after receiving the shipment.	Verifies that stock increased by exactly the quantity left over after the waitlist was served (received quantity − total quantity filled). If the waitlist used up the whole shipment, stock is unchanged.
Product waitlist	Run Query Product Waitlist and note all current waitlist entries and quantities.	Run Query Product Waitlist again after processing the shipment.	Verify that fully filled requests were removed and partially filled requests were updated with the remaining quantity.
Client balance	Run Query Client Details for each affected client and note the current balance.	Run Query Client Details again after processing the shipment.	Verify that each client served was debited by filledQty × unit price, and that clients not served are unchanged.
Client orders	Run Query Client Orders and note the client's existing orders.	Run Query Client Orders again after processing the shipment.	Verify that a new order was created for each client served, showing the product, the quantity filled, and the unit price.
Unfilled quantities	Run Query Product Waitlist and Product Details to record existing waitlisted quantities and stock.	Run Query Product Waitlist and Product Details again after processing.	Verify that any quantity that could not be filled remained on the waitlist while available quantity was correctly allocated.

Example Test:

Before shipment: Product P1 has stock 0 and a unit price of $8.00. The P1 waitlist is Client A × 5, then Client B × 3. Both clients have a $0.00 balance.

Test 1: the clerk receives 7 units of P1. Client A is filled completely (5 units, $40.00) and removed from the waitlist. Client B is filled partly (2 units, $16.00), and B's remaining 1 unit stays on the waitlist. Nothing is left over, so stock stays at 0.

Query	Expected result after Test 1
Query Product Details (P1)	Stock 0 (unchanged; the whole shipment went to the waitlist).
Query Product Waitlist (P1)	One entry: Client B, quantity 1. Client A has been removed.
Query Client Details (A, B)	A: balance $40.00 (was $0.00). B: balance $16.00 (was $0.00).
Query Client Orders (A, B)	A: new order, P1 × 5 at $8.00 = $40.00. B: new order, P1 × 2 at $8.00 = $16.00.

Test 2: starting from the same state, the clerk receives 10 units of P1. Client A gets 5 ($40.00) and Client B gets 3 ($24.00); both are removed from the waitlist, and the remaining 2 units are added to stock.

Query	Expected result after Test 2
Query Product Details (P1)	Stock 2 (was 0).
Query Product Waitlist (P1)	"No clients are currently waitlisted."
Query Client Details (A, B)	A: balance $40.00. B: balance $24.00.
Query Client Orders (A, B)	A: new order, P1 × 5 at $8.00 = $40.00. B: new order, P1 × 3 at $8.00 = $24.00.

Question 2: Detailed Use Cases

Actor: Clerk (trusted). Preconditions: the product exists; every client on its waitlist exists. Postconditions: each waitlisted request is filled as far as the shipment allows; a new order is stored and debited for each client served; fully filled entries are removed from the waitlist and partly filled entries keep the unfilled quantity; any leftover quantity is added to stock. No payment is processed.

Actor	System Response
1. Request to receive a shipment.	2. Ask for product ID.

Actor	System Response
3. Enter product ID.	4. Search for the product. If the product does not exist, display a message and exit the operation.
	5. Ask for quantity received.
6. Enter quantity received.	7. Validate that the quantity is a positive whole number. If invalid, display an error and ask again.
	8. Retrieve the product's waitlist. Set remainingQty to the quantity received.
	9. Select the next WaitlistItem. If there are no more entries, go to step 17.
	10. Determine filledQty = min(remainingQty, waitlistedQty).
	11. Create a new Order for the client with a unique order ID and the current date, containing the product, filledQty, and the unit price.
	12. Debit the client's account by the order total (filledQty × unit price) and store the order with the client.
	13. If filledQty equals waitlistedQty, remove the WaitlistItem. Otherwise, reduce the WaitlistItem quantity by filledQty.
	14. Reduce remainingQty by filledQty.
	15. If remainingQty > 0 and additional WaitlistItems exist, repeat steps 9–14.
	16. Continue until either the end of the waitlist is reached or remainingQty becomes 0.
	17. If remainingQty > 0, add the remaining quantity to product stock.
	18. Save all updates to stock, waitlists, client balances, and orders.
	19. Display a summary showing each client served, the quantity filled, the amount debited, and the quantity added to stock. Display “Shipment processed” and exit the operation.

In Receive Shipment, we have four queries that support the Table found in Question 1: Product Details verifies stock changes, Product Waitlist verifies waitlist changes, Client Details verifies account balance changes, and Client Orders verifies orders were created.

Query Product Details

Actor	System Response

1. Request product details.	2. Ask for product ID.
3. Enter product ID.	4. Search for the product. If the product does not exist, display a message and exit the operation.
	5. Display the product ID, product name, price, and stock quantity.

Query Product Waitlist

Actor	System Response
1. Request product waitlist.	2. Ask for product ID.
3. Enter product ID.	4. Search for the product. If the product does not exist, display a message and exit the operation.
	5. Retrieve the product's waitlist.
	6. If the waitlist is empty, display “No clients are currently waitlisted.” Otherwise, display each client ID and quantity currently on the waitlist.

Query Client Details

Actor	System Response
1. Request client details.	2. Ask for client ID.
3. Enter client ID.	4. Search for the client. If the client does not exist, display a message and exit the operation.
	5. Display the client ID, name, address, and current account balance.

Query Client Orders

Actor	System Response
1. Request client orders.	2. Ask for client ID.
3. Enter client ID.	4. Search for the client. If the client does not exist, display a message and exit the operation.
	5. Retrieve the client's orders.
	6. If the client has no orders, display “No orders found.” Otherwise, display the order ID, date, products, quantities, unit prices, and order total for each order.

Alternate Flow Examples

Alternative Flow A1: Product Not Found

Actor	System Response
A1.1 Enter product ID.	A1.2 Search for the product.
	A1.3 Product not found.
	A1.4 Display "Product not found."
	A1.5 Operation terminates.

Alternative Flow A2: Invalid Quantity

Actor	System Response
A2.1 Enter shipment quantity.	A2.2 Validate quantity.
	A2.3 Quantity is not a positive whole number.
	A2.4 Display "Invalid quantity entered."
	A2.5 Request shipment quantity again.

Alternative Flow A3: Empty Waitlist

Actor	System Response
A3.1 Enter valid product ID and shipment quantity.	A3.2 Retrieve product waitlist.
	A3.3 Waitlist is empty.
	A3.4 Add entire shipment quantity to product stock.
	A3.5 Save updates.
	A3.6 Display confirmation message.

Alternative Flow A4: Partial Fulfillment

Actor	System Response
A4.1 Enter valid product ID and shipment quantity.	A4.2 Process waitlist entry.
	A4.3 Available shipment quantity is less than requested quantity.

Actor	System Response
	A4.4 Create a new order for the available quantity.
	A4.5 Debit client account for quantity supplied.
	A4.6 Update WaitlistItem with remaining quantity.
	A4.7 Continue processing or terminate if shipment quantity becomes zero.

Alternative Flow A5: Shipment Completely Consumed by Waitlist

Actor	System Response
A5.1 Enter valid product ID and shipment quantity.	A5.2 Process all available shipment quantity against waitlist entries.
	A5.3 Shipment quantity becomes zero before waitlist is empty.
	A5.4 No stock is added to inventory.
	A5.5 Remaining requests stay on the waitlist.
	A5.6 Display confirmation message.

Question 3: Design a solution for the three features listed above, and for all queries identified in Question 2

The work is split into two parts. First, UserInterface gets and checks the clerk's input. Second, Warehouse.receiveShipment(productId, receivedQty) processes the waitlist and stock. The user interface communicates only with Warehouse, which acts as the facade.

Step	Description	Responsible class.method	Design Choice
1	Receive the shipment request from the clerk and ask for the product ID.	UserInterface	The UserInterface handles all interaction with the clerk.
2	Look up the product.	UserInterface → Warehouse.getProduct(productId) → ProdList.search(productId)	The UI talks only to Warehouse. ProdList owns the product collection, so it performs the search. A null result ends the operation before any data changes.

Step	Description	Responsible class.method	Design Choice
3	Request and validate the quantity received.	UserInterface	Input checks belong in the UI. Only a positive whole number is passed on, so Warehouse never receives invalid input.
4	Request shipment processing.	UserInterface → Warehouse.receiveShipment(productId, receivedQty)	Warehouse is the facade and coordinates the whole operation, so stock, waitlist, order, and balance updates happen together.
5	Search for the product.	Warehouse → ProdList.search(productId)	ProdList is responsible for locating Product objects.
6	Retrieve the product's waitlist.	Product.getWaitlist() returns an iterator over WaitlistItems	The waitlist belongs to the Product because clients are waiting for that specific product. Entries are served in the order they were added.
7	Get the next waitlist entry’s client and quantity.	WaitlistItem.getClient(); WaitlistItem.getQuantity()	WaitlistItem stores the client and quantity requested, so no ClientList search is needed.
8	Calculate quantity to fill.	Warehouse: filledQty = min(remainingQty, waitlistedQty)	Warehouse coordinates fulfillment decisions using the shipment quantity still available.
9	Create a new order for the filled quantity.	new Order(client, date); Product.getPrice(); Order.addItem(product, filledQty, price)	A new Order is always created, the same as in Process Client’s Order, so past orders are never changed. The unit price is copied into the order line.
10	Debit the client's account and store the order.	Order.getTotal(); Client.debit(total); Client.addOrder(order)	Financial information belongs to the Client object. Debit updates the balance only; no payment is processed.
11	Update or remove the waitlist entry.	If filledQty = waitlistedQty: Product.removeFromWaitlist(waitlistItem). Otherwise: WaitlistItem.setQuantity(waitlistedQty − filledQty)	Product owns the waitlist, so it removes entries. WaitlistItem owns its quantity, so it changes it.
12	Repeat for additional waitlist entries until quantity is exhausted or waitlist is empty.	Warehouse: remainingQty = remainingQty − filledQty; loop to step 7	Warehouse controls the processing loop.
13	Add remaining received quantity to stock.	Product.addStock(remainingQty), only if remainingQty > 0	Inventory information belongs to the Product, so only Product changes stock.

Step	Description	Responsible class.method	Design Choice
14	Return and display the result.	Warehouse returns a shipment summary → UserInterface displays it	The clerk sees each client served, the quantities filled, the amounts debited, and the quantity added to stock.

Queries Used to Verify Receive Shipment

Query	Steps and responsible class.method	Design choice
Query Product Details	UserInterface → Warehouse.getProduct(productId) → ProdList.search(productId) → UI displays product ID, name, price, and stock quantity using Product getters.	ProdList owns the product collection. This query verifies the final stock quantity after shipment processing.
Query Product Waitlist	UserInterface → Warehouse.getWaitlist(productId) → ProdList.search(productId) → Product.getWaitlist() → UI iterates WaitlistItems and displays client ID and quantity.	The waitlist belongs to Product. This query verifies that fully filled entries are removed and partially filled entries are updated correctly. It also verifies that unfilled quantities stayed on the waitlist.
Query Client Details	UserInterface → Warehouse.getClient(clientId) → ClientList.search(clientId) → UI displays client ID, name, address, and balance using Client getters.	Read-only query. Comparing balances before and after shipment verifies that clients were debited for filled quantities.
Query Client Orders	UserInterface → Warehouse.getOrders(clientId) → ClientList.search(clientId) → Client.getOrders() → UI displays order ID, date, line items, quantities, unit prices, and total using Order methods.	Orders are stored with the client. The query verifies that a new order was created when a waitlisted request was filled.

Unfilled quantities (Question 1) are checked with Query Product Waitlist together with Query Product Details, so a separate query is not needed.

Question 4: Creating Sequence Diagrams

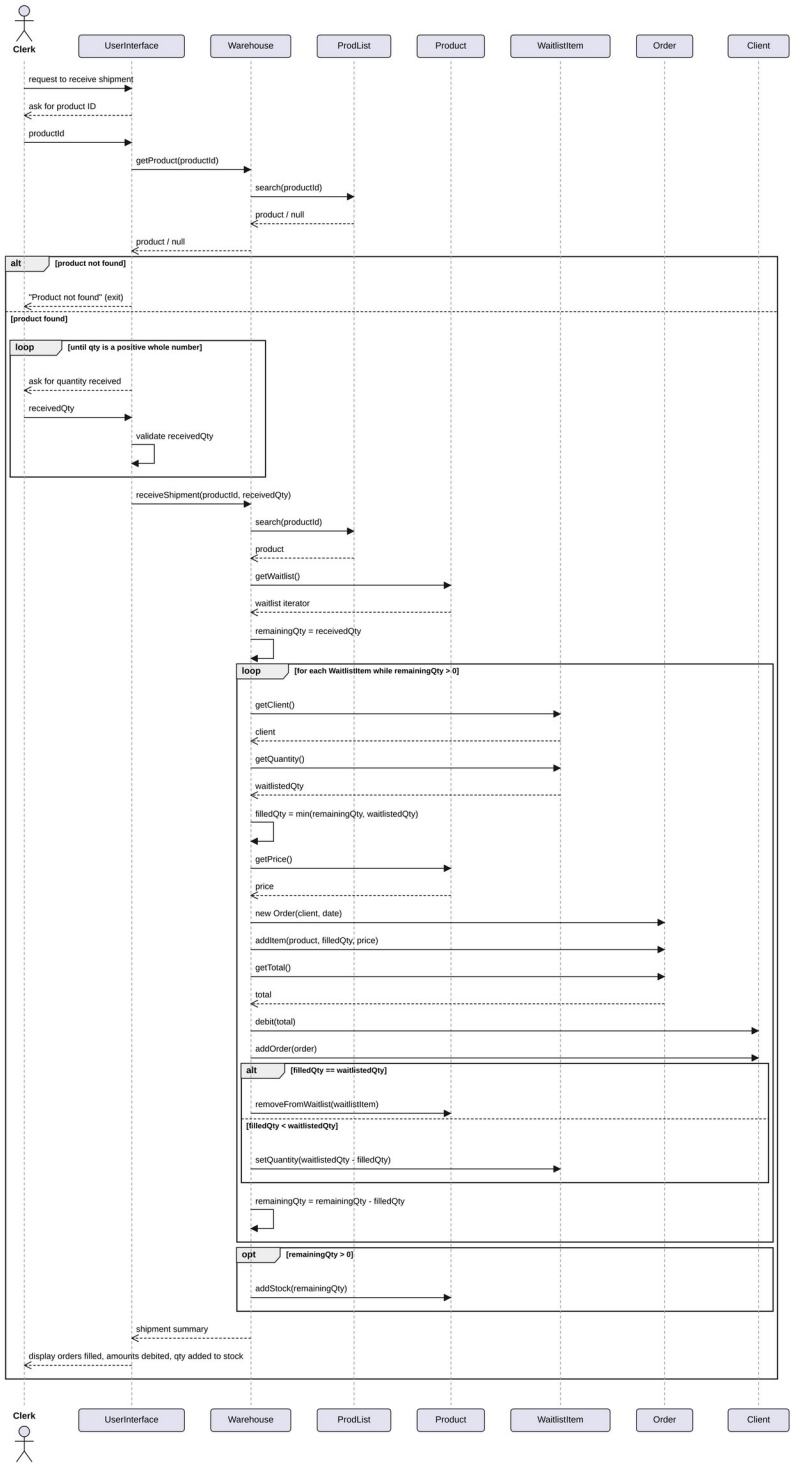


Figure 1: Receive Shipment

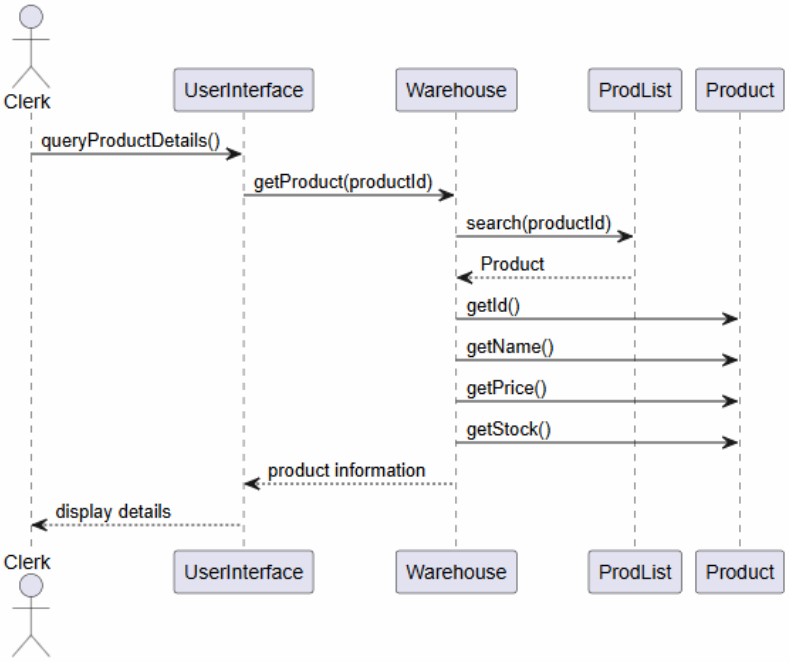


Figure 2: Query Product Details

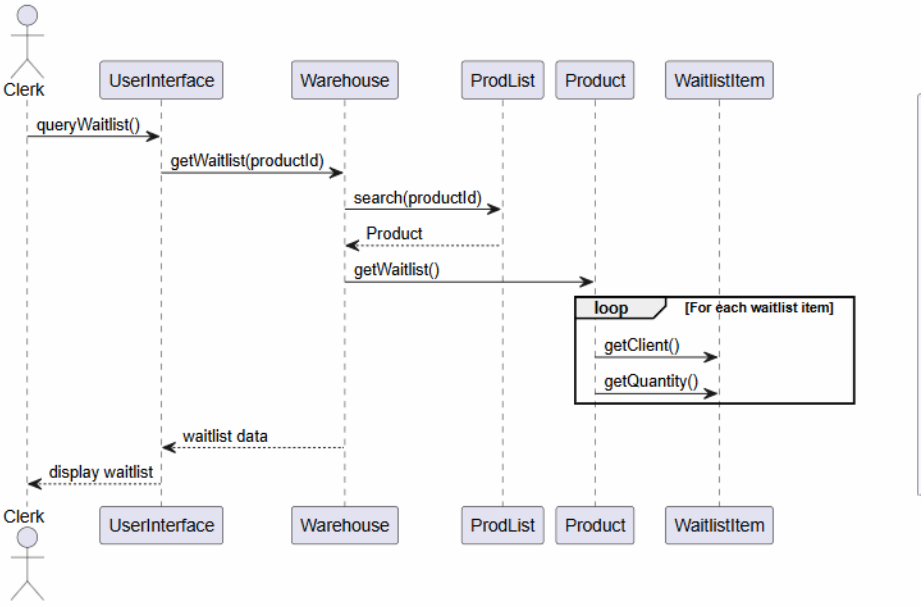


Figure 3: Query Product Waitlist

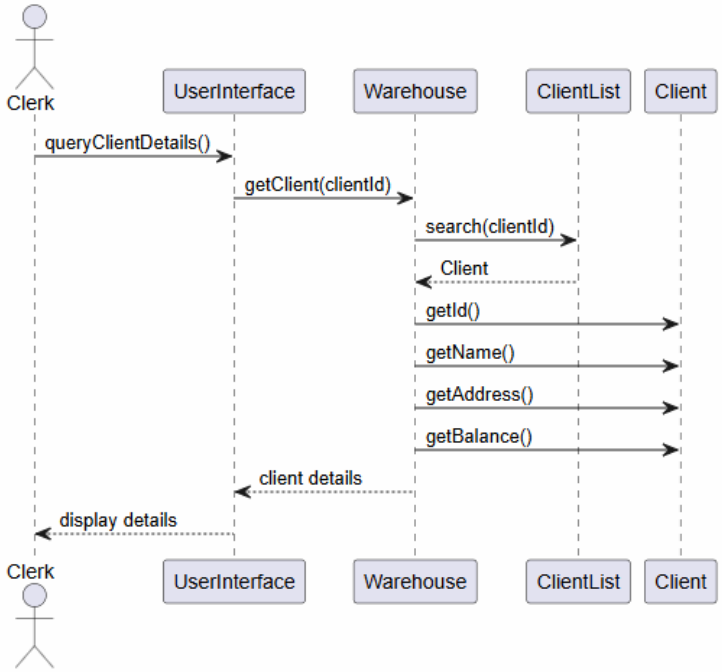


Figure 4: Query Client Details

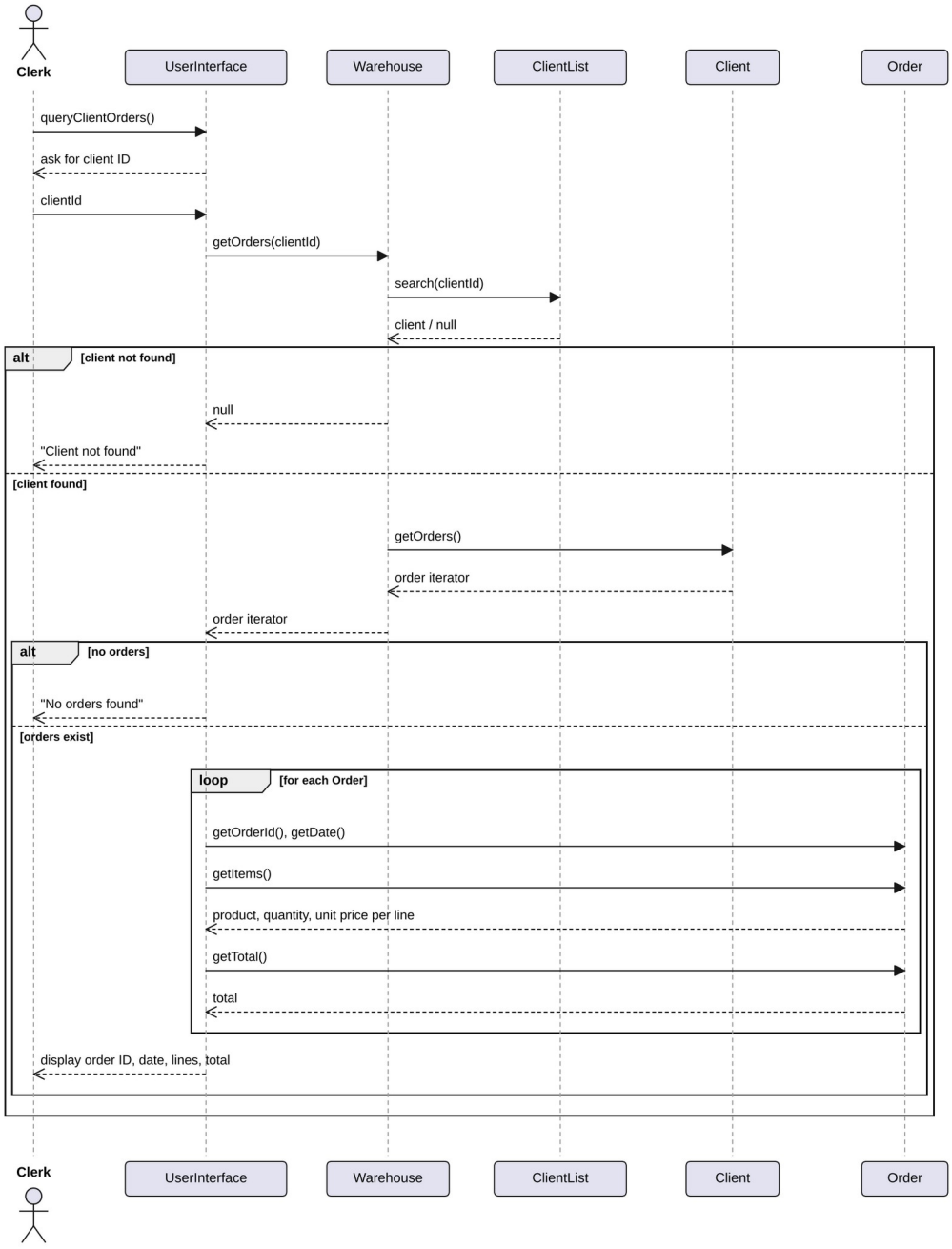


Figure 5: Query Client Orders